

Luiss

Libera Università Internazionale
degli Studi Sociali Guido Carli

StringsIntroduction to computer programmingManagement an

Introduction to Computer Programming

Management and Artificial Intelligence

LUISS



Introduction to Strings

Recall from past lectures that:

- Strings are objects that can host letters, numbers, spaces, special characters.

More in detail:

- strings are among the most popular types in Python
- we can create strings simply by enclosing characters in quotes
- Python treats single quotes as the same as double quotes (*Caveat: if your string contains a quote (e.g., `let's dance`), enclose the string in double quotes. Similarly, if your string contains double quotes (e.g., `I say "hello"`), enclose the string in single quotes.*)
- to create a string, just perform a variable assignment

Examples:

```
var1 = 'Hello World!'
var2 = "Python is cool! (hopefully)"

print(var1)
print(var2)
```

```
Hello World!
Python is cool! (hopefully)
```

- Python **does not support a character type**: chars are treated as strings of length 1, thus also considered a substring
- To access substrings, use the **square brackets for slicing** along with the **index or indices** to obtain your substring

Definition The slice is a segment of the string. It can be gathered as
`string_name[start:stop]`

NOTE: Note that we include `start` in the slice, but we only go up to (and **not including**) `stop`. Further, indexing is **0-based** (the 1st element has index 0, the 2nd element has index 1 and so on ...).

Examples:

```
var1 = 'Hello World!'
var2 = "Python is cool! (hopefully)"

# indexing
print(f'var1[0] = {var1[0]}')

# slicing
print(f'var2[1:5] = {var2[1:5]}')
```

```
var1[0] = H
var2[1:5] = ytho
```

Slicing

Syntax of slicing:

- `s[start:stop]` : chars `start` through `stop-1`
- `s[start:]` : chars `start` through the rest of the string
- `s[:stop]` : chars from the beginning through `stop-1`
- `s[:]` : a full copy of the string

There is also the `step` value, which can be used with any of the above:

- `s[start:stop:step]` : `start` through not past `stop`, by `step`

`start` and `stop` can be negative numbers: this means that Python starts counting from the end of the string

- `s[-1]` : last char in the string
- `s[-2:]` : last two chars in the string
- `s[:-2]` : everything except the last two chars

Examples:

```
# Some examples on slicing:  
s = 'hello'  
print(f"s[1:4] -> {s[1:4]}")  
print(f"s[1:] -> {s[1:]}")  
print(f"s[:] -> {s[:]} (overestimating the upper bound)")  
print(f"s[1:1000] -> {s[1:1000]}")  
print(f"s[-1] -> {s[-1]} (reverse indexing)")  
print(f"s[::] -> {s[::]}")  
print(f"s[:-3] -> {s[:-3]} (everything except the last three chars)")
```

```
s[1:4] -> ell  
s[1:] -> ello  
s[:] -> hello (overestimating the upper bound)  
s[1:1000] -> ello  
s[-1] -> o (reverse indexing)  
s[::] -> hello  
s[:-3] -> he (everything except the last three chars)
```

Operations on Strings

Recall from Lecture 4 the two operators:

- **Concatenate:** with the `+` operator
- **Replicate:** with the `*` operator

Examples:

```
print(f'Hello " + "World!" -> {"Hello " + "World!"}')  
print(f'Python'*3 -> {"Python"*3}')
```

```
"Hello " + "World!" -> Hello World!  
"Python"*3 -> PythonPythonPython
```


More operations on strings

Let `a = "Hello"` and `b = "Python"`.

Operator	Description	Example
<code>+</code>	Concatenation: adds values on either side of the operator	<code>a+b</code> yields <code>"HelloPython"</code>
<code>*</code>	Replication: creates new strings concatenating multiple copies of the original string	<code>a*2</code> yields <code>"HelloHello"</code>
<code>[]</code>	Slice: gives the character from the given index	<code>a[1]</code> yields <code>"e"</code>
<code>[:]</code>	Range: gives the character(s) from the given range	<code>a[1:4]</code> yields <code>"ell"</code>
<code>==</code>	Equality check: yields <code>True</code> if two strings are equal	<code>"Hello" == a</code> yields <code>True</code>
<code>in</code>	Membership: yields <code>True</code> if a substring appears in the string	<code>"He" in a</code> yields <code>True</code>
<code>not in</code>	Membership: yields <code>True</code> if a substring does not appear in the string	<code>"Me" not in a</code> yields <code>True</code>

Iterating over Strings

You can iterate over strings (i.e., get one character at the time) via `for` or `while` loops:

Note `len(s)` is a built-in function that returns the length of the string `s`.

```
# Example: print each letter of s  
s = "Python Program"  
for letter in s:  
    print(letter, end=' ')
```

P y t h o n P r o g r a m

```
# Example: print each letter of s using while loop  
s = "Python Program"  
i = 0  
while i < len(s):  
    print(s[i], end=' ')  
    i = i + 1
```

P y t h o n P r o g r a m

```
# Example: count the number of times 'a' appears in s
s = "Python Program"
count = 0
for letter in s:
    if letter == 'a':
        count = count + 1
print(f"Count of 'a': {count}")
```

Count of 'a': 1

```
# Example: count the number of times 'p' appears in s
s = "Python Program"
count = 0
i = 0
while i < len(s):
    if s[i] == 'p':
        count = count + 1
    i = i + 1
print(f"Count of 'p': {count} (because 'p' != 'P')")
```

Count of 'p': 0 (because 'p' != 'P')

Changing Strings

Strings are **immutable** sets of characters:

- we cannot change any character within a string once it is created

```
# Example: strings are immutable  
s = "Python Program"  
print(s[1])
```

y

```
try:  
    s[1] = "a" # ERROR!!!  
except TypeError as e:  
    print(e)
```

'str' object does not support item assignment

So, how can we update strings?

- (re)assign a variable with a new string
- the new value can be related to the previous value or to a completely different string altogether

For instance:

```
var1 = "Hello World"
var2 = var1[:6] + "Python"
print(f'var1 = {var1}')
print(f'var2 = {var2}')
var1 = var2 + "!"
print(f'var1 (updated) = {var1}')
print(f'var2 (unchanged) = {var2}')
```

```
var1 = Hello World
var2 = Hello Python
var1 (updated) = Hello Python!
var2 (unchanged) = Hello Python
```

Built-in String Methods

Python has a set of built-in methods that you can use on strings.

You can find the full list at <https://docs.python.org/3/library/stdtypes.html#string-methods>

Warning All string methods return new values. They do not change the original string.

`str.capitalize()`

`s.capitalize()` returns a copy of the string with its first character capitalised and the rest lowercased.

```
s = "supercalifragilisticexpialidocious"  
print(s.capitalize())
```

Supercalifragilisticexpialidocious

```
s = "SupercaliFragilisTicexpialid0ciouS"  
print(s.capitalize())
```

Supercalifragilisticexpialidocious

```
# if the string is already capitalised, it has no effect  
s = "Supercalifragilisticexpialidocious"  
print(s.capitalize())
```

Supercalifragilisticexpialidocious

`str.lower()` and `str.upper()`

`s.lower()` returns a copy of the string with all the cased characters converted to lowercase.
`s.upper()` returns a copy of the string with all the cased characters converted to uppercase.

```
s = "SUPercalifragiliSTIcexpialidocious"  
print(f'lower: {s.lower()}')  
print(f'upper: {s.upper()}')
```

```
lower: supercalifragilisticexpialidocious  
upper: SUPERCALIFRAGILISTICEXPIALIDOCIOUS
```

```
s = "supercalifragilisticexpialidocious"  
print(f'already lower: {s.lower()}')
```

```
already lower: supercalifragilisticexpialidocious
```

```
s = "SUPERCALIFRAGILISTICEXPIALIDOCIOUS"  
print(f'already upper: {s.upper()}')
```

```
already upper: SUPERCALIFRAGILISTICEXPIALIDOCIOUS
```


`str.count()`

`s.count(sub, [start, end])` returns the number of non-overlapping occurrences of substring `sub` in the string `s`. Optionally, you may specify `start` and `end` to look for `sub` in a slice of `s`.

```
s = "supercalifragilisticexpialidocious"  
print(f"s.count('li'): {s.count('li')}")
```

```
s.count('li'): 3
```

```
print(f"s.count('LI'): {s.count('LI')}")
```

```
s.count('LI'): 3
```

```
print(f"s.count('li', 0, 10): {s.count('li', 0, 10)}")
```

```
s.count('li', 0, 10): 1
```

```
print(f"s[0:10].count('li'): {s[0:10].count('li')}")
```

```
s[0:10].count('li'): 1
```

`str.startswith()`

`s.startswith(sub, [start, end])` returns `True` if the string *starts* with the specified `sub`, otherwise returns `False`.

Use optional parameters `start` and `end` to test a slice of `s`.

```
s = "supercalifragilisticexpialidocious"  
print(f"s.startswith('super'): {s.startswith('super')}")
```

```
s.startswith('super'): True
```

```
print(f"s.startswith('super', 1, 10): {s.startswith('super', 1, 10)} (slice is 'upercalif')")
```

```
s.startswith('super', 1, 10): False (slice is 'upercalif')
```

```
print(f"s.startswith('uper', 1, 10): {s.startswith('uper', 1, 10)}")
```

```
s.startswith('uper', 1, 10): True
```

`str.endswith()`

`s.endswith(sub, [start, end])` returns `True` if the string *ends* with the specified `sub`, otherwise returns `False`.

```
print(f"s.endswith('ous'): {s.endswith('ous')}")
```

```
s.endswith('ous'): True
```

```
print(f"s.endswith('calif', 1, 10): {s.endswith('calif', 1, 10)}")
```

```
s.endswith('calif', 1, 10): True
```

`str.find()` and `str.index()`

`s.find(sub, [start, end])` returns the lowest index in the string `s` where substring `sub`. If `sub` is not found, it returns `-1`. `s.index(sub, [start, end])` works like `s.find()`, but raises `ValueError` when the substring is not found.

HINT: Use `find()` or `index()` if you need the position of the substring. To check if `sub` exists in `s`, you may use the `in` operator.

```
s = "supercalifragilisticexpialidocious"  
print(f"s.find('calif'): {s.find('calif')}")
```

```
s.find('calif'): 5
```

```
print(f"s.index('calif'): {s.index('calif')}")
```

```
s.index('calif'): 5
```

```
print(f"s.find('calio'): {s.find('calio')}")
```

```
s.find('calio'): -1
```

```
try:  
    s.index('calio')  
except ValueError as e:  
    print(f"s.index('calio'): {e}")
```

s.index('calio'): substring not found

```
print(f"'calif' in s: {'calif' in s}")
```

'calif' in s: True

`str.is*()`

- `s.islower()` : returns `True` if a non-empty string `s` contains only lowercase characters.
- `s.isupper()` : returns `True` if a non-empty string `s` contains only uppercase characters.
- `s.isalnum()` : returns `True` if a non-empty string `s` contains only alphanumeric characters (numbers and letters).
- `s.isalpha()` : returns `True` if a non-empty string `s` contains only alphabetic characters (letters).
- `s.isdecimal()` : returns `True` if a non-empty string `s` contains only decimal numbers.
- `s.isdigit()` : returns `True` if a non-empty string `s` contains only digits (decimals, subscripts and superscripts).
- `s.isnumeric()` : returns `True` if a non-empty string `s` contains only numeric characters (decimals, subscripts, superscripts, vulgar fractions, roman numerals, currency numerators).
- `s.istitle()` : returns `True` if a non-empty string `s` contains is titlecased.

Exercise Session

After covering the lists, check out the exercises: [here](#).

